JUNIOR FC STORE

Resumen de la entrega: IA, agente y funcionalidad asíncrona

Qué se construyó, por qué se tomó cada decisión y cómo cumple cada criterio de la rúbrica. Todo está desplegado en AWS (us-west-2) y OpenShift, y se probó en vivo el 4 de noviembre de 2026.

1. Qué se pidió

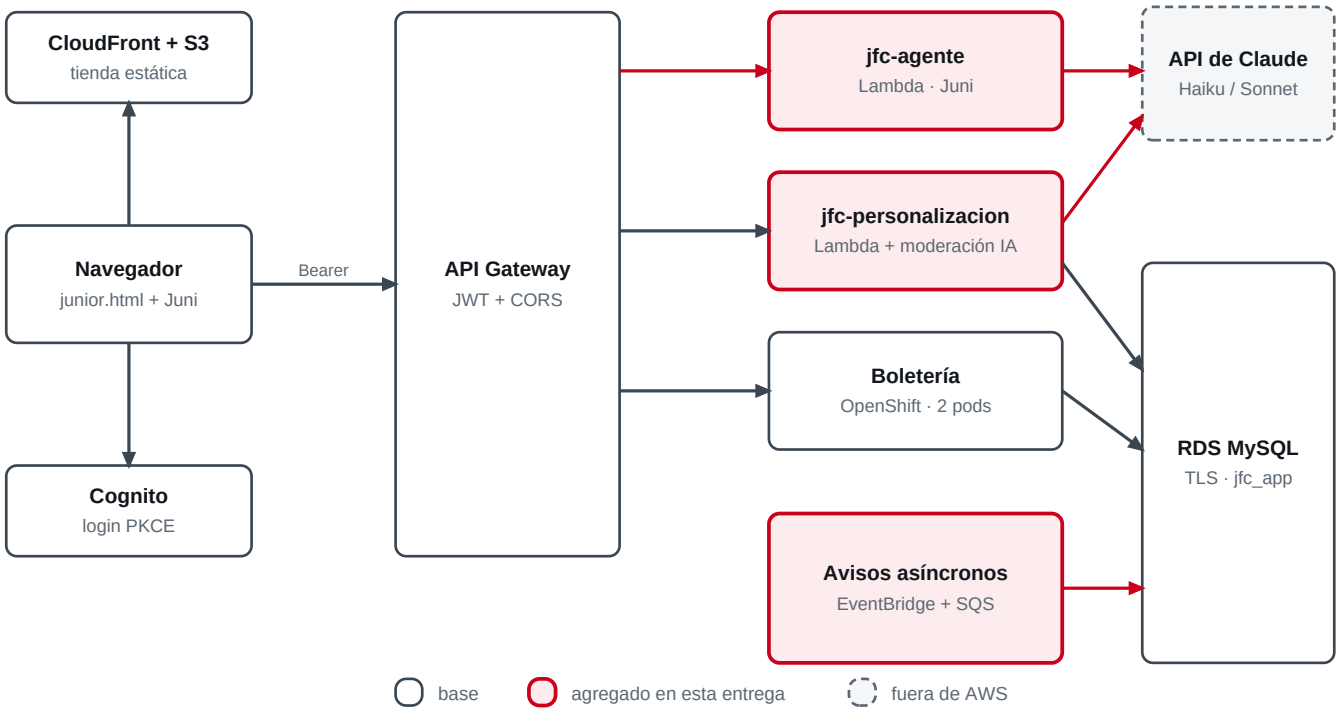
La actividad pide extender la tienda del Junior FC con tres funcionalidades nuevas y recuerda que los recursos solo deben estar encendidos mientras se desarrolla o se presenta (RDS cobra por hora y EKS no se usa). Las boletas deben seguir el mismo patrón que las camisetas: una parte ligera en una Lambda y una parte pesada en un contenedor.

Criterio de la rúbrica	Peso	Lo que construimos
Implementación de funcionalidad IA	40%	Moderación con IA del nombre que se imprime en la camiseta
Agente con su conexión a los servicios internos	30%	Juni, un agente de texto que reserva y paga boletas
Funcionalidad asíncrona	30%	Aviso de reserva por vencer (EventBridge, SQS, correo y grupo de la clase)

2. Punto de partida

La tienda ya tenía una página estática en CloudFront y S3, login con Cognito (Hosted UI con PKCE), un API Gateway con autorizador JWT, una Lambda para personalizar camisetas, un servicio de boletería (FastAPI en Docker, 2 pods en OpenShift) y una base MySQL en RDS. Elegimos extender ese producto en lugar de crear uno nuevo, como pidió la actividad.

3. Arquitectura actual



El navegador solo habla con API Gateway, que valida el token de Cognito antes de que corra cualquier backend. El agente vuelve a entrar por el mismo gateway con el token del usuario, así que pasa por las mismas reglas que el navegador. La versión interactiva, con el detalle de cada flujo, está en el artifact «Arquitectura Junior FC».

4. Funcionalidad 1: moderación de nombres con IA (40%)

Necesidad real. Un hincha puede escribir cualquier cosa para imprimirla en una camiseta oficial. La lista de palabras bloqueadas atrapa lo conocido, pero no puede prever cada insulto, el nombre de un club rival, una marca o una cadena sin sentido. Un modelo lee el nombre como lo haría una persona de la tienda.

Cómo funciona. Corre al final, después de las pruebas gratuitas (lista de palabras, formato, dorsal reservado y parche), así que solo ve nombres que ya pasaron lo barato. Claude Haiku 5.5 responde con una llamada a herramienta forzada, de modo que el veredicto siempre es un dato estructurado: PERMITIDO, OFENSIVO, ODIO, SEXUAL, VIOLENCIA_O_DROGAS, POLITICA_O_RELIGION, CLUB_RIVAL, MARCA_COMERCIAL o SIN_SENTIDO.

Decisiones y por qué.

- **El mensaje al hincha sale de una tabla fija del código, nunca del modelo.** El nombre es texto no confiable: un modelo podría repetir la palabra bloqueada o dejarse convencer de lo contrario. El nombre viaja entre etiquetas como dato, no como instrucción.
- **Si la IA falla, el nombre pasa.** Una caída del modelo no debe impedir que la tienda venda camisetas; la lista y el formato ya corrieron.
- **Ante la duda, permitido.** Bloquear por error a un hincha cuesta más que dejar pasar un apodo inocente. Por eso se ajustó la regla cuando «URIBE» salió bloqueado como política: un apellido suelto se permite, un lema no.

- **Caché por contenedor** (máximo 256 nombres) para no pagar dos veces por el mismo apellido.

Evidencia. Con la API real: GARCÍA, TUTA, CAÑAS, TEO, URIBE y SANTOS pasan; MILLONARIOS y NACIONAL salen como club rival; NIKE como marca; HIJUEPUTA, M13RD4 y «PUTO EL QUE LEA» como ofensivos; ASDFGH como sin sentido; NAZI como odio. Cada consulta tarda entre 1 y 5 segundos. También se probó contra la Lambda ya desplegada, y el guardado de personalizaciones funciona desde la tienda.

5. Funcionalidad 2: Juni, el agente de compra (30%)

Necesidad real. Comprar boletas exige varias decisiones (partido, tribuna, cuántas sillas, pago). Juni conversa, pregunta lo que falta y ejecuta las acciones del portal por el usuario.
Cómo funciona. Una Lambda (`jfc-agente`) repite el ciclo «el modelo pide una herramienta, la Lambda la ejecuta» con Claude Sonnet 5.5, hasta que el modelo responde con texto. Hay cinco herramientas: listar partidos, ver disponibilidad, reservar sillas, confirmar la compra y consultar las boletas del usuario. Cada herramienta es una llamada HTTP a los servicios internos de la boletería a través de API Gateway.
Decisiones y por qué.

- **Usa el token del propio usuario.** El agente no tiene permisos propios: no puede hacer nada que el usuario no pueda hacer desde la página.
- **El pago exige un «sí» en un mensaje posterior a la reserva.** Esta regla está en el código (error FALTA_CONFIRMACION), no en el prompt; un modelo no puede saltársela aunque lo convenzan.
- **Servidor sin estado.** El navegador guarda y reenvía el historial; la Lambda lo filtra y recorta (máximo 40 mensajes y 60 KB). No hay sesiones que limpiar ni conversaciones guardadas en el servidor.
- **Límites de costo y abuso:** máximo 6 pasos por turno y 2 peticiones por segundo en la ruta del agente.

Evidencia. Juni lista los partidos, convierte la hora a Barranquilla, pregunta tribuna y cantidad, reserva sillas juntas y pide confirmación antes de cobrar. Funciona en vivo desde la tienda contra la boletería real en OpenShift.

6. Funcionalidad 3: aviso asíncrono de reserva por vencer (30%)

Necesidad real. Una reserva de sillas se libera a los 10 minutos. Quien se distrae pierde sus sillas sin enterarse. El aviso no debe retrasar la compra, por eso no corre dentro de ella.
Cómo funciona. EventBridge dispara cada minuto una Lambda escáner que busca reservas en sus últimos 2 minutos, las marca en la tabla `aviso_reserva` y encola un mensaje por reserva en SQS. Una segunda Lambda, el notificador, lee la cola, confirma que la reserva sigue retenida y envía el aviso por correo (SES) y al grupo de la clase (webhook). Tras tres fallos, el mensaje pasa a una cola de fallos.
Decisiones y por qué.

- **Cola SQS entre las dos Lambdas.** Si el correo o el webhook fallan, el mensaje se reintenta sin repetir el escaneo, y los casos irrecuperables quedan guardados para revisión.
- **Ventana de 2 minutos con escaneo cada minuto.** Garantiza ver cada reserva al menos una vez; la tabla evita avisar dos veces.
- **El notificador vuelve a verificar la reserva.** Si ya se pagó o se liberó mientras el mensaje esperaba, se descarta: nadie recibe un aviso falso.
- **Se puede apagar** con un parámetro (AvisosActivos) para que no corra cuando no se está presentando.

Estado honesto. Las dos Lambdas, la cola, la cola de fallos y la regla de EventBridge están desplegadas y el escáner corre sin errores cada minuto. Falta configurar el destino real: un remitente verificado en SES y la

URL del webhook del grupo de la clase. Mientras tanto el aviso queda en los registros de CloudWatch. La prueba de punta a punta (reservar, esperar 8 minutos y ver el aviso) está pendiente.

7. Cómo cumple cada criterio de la rúbrica

Criterio	Por qué cumple	Dónde verlo
IA (40%) **La IA es parte esencial, no un chatbot**	Clasifica texto libre del usuario para resolver una necesidad real: qué se puede imprimir en una camiseta oficial. Sin el modelo, la tienda dependería de una lista incompleta. El veredicto es estructurado y el mensaje al usuario es controlado.	Guardar una personalización con MILLONARIOS, NIKE o ASDFGH; Lambda jfc-personalizacion
Agente (30%) **Pregunta y llama a los servicios**	Conversa para reunir partido, tribuna y cantidad, llama a las rutas de la boletería con el token del usuario y no paga sin confirmación explícita.	Widget de Juni en la tienda; Lambda jfc-agente; logs con las herramientas usadas
Asíncrono (30%) **Notificado al grupo de la clase**	Proceso desacoplado con EventBridge, SQS, dos Lambdas y cola de fallos. Entrega por correo y por webhook al grupo (por configurar).	Consolas de EventBridge y SQS; logs de jfc-avisos-notificador
Recursos solo encendidos al usar	Todo es por uso salvo RDS, que se apaga con un comando. No se usa EKS: la parte pesada corre en el Sandbox gratuito de OpenShift.	scripts/rds.sh stop; Cost Explorer

8. Decisiones transversales

Decisión	Por qué
Combo equilibrado (moderación, agente, aviso)	Cada funcionalidad cubre un criterio distinto, reutiliza la infraestructura existente y se puede probar en vivo en minutos.
API de Claude en vez de Bedrock	Esta cuenta de AWS no tiene acceso ni cuotas de Bedrock. Un adaptador traduce la misma llamada al formato de la API de Claude; si algún día se habilita Bedrock basta cambiar una variable de entorno.
Lambdas y servicios por uso, RDS apagable	La actividad exige no dejar recursos encendidos. RDS se detiene con scripts/rds.sh y se restaura desde un snapshot si hace falta.
Boletas igual que las camisetas	Parte ligera en Lambda (personalización) y parte pesada en contenedor (boletería), ambas detrás del mismo API Gateway y la misma base.
Infraestructura como código (SAM)	Todo el backend se crea con una plantilla; el despliegue es repetible y las claves se pasan como parámetros sin imprimirse.
Pruebas automáticas con dobles	128 pruebas (120 del backend y 8 de la boletería) corren sin AWS ni red, con un modelo simulado.

9. Problemas encontrados y cómo se resolvieron

Problema	Causa	Solución
Bedrock inutilizable	La cuenta no tiene acceso a modelos ni cuotas	Adaptador a la API de Claude con la misma forma de llamada; las pruebas existentes siguieron intactas
«URIBE» bloqueado como política	El prompt trataba cualquier apellido de político como consigna	Regla ajustada: un apellido suelto se permite, un lema no
RDS no existía al volver a desplegar	La instancia se había eliminado para ahorrar costos	Restaurada desde el snapshot final con el mismo endpoint
La app de OpenShift había desaparecido	El Sandbox se reinició por inactividad	Se recrearon Secret, Deployment, Service y Route con los YAML del repositorio
Error de «CORS» al guardar la personalización	Faltaban en API Gateway las rutas GET y POST /personalizaciones (borradas fuera de CloudFormation)	Recreadas con autorización JWT; el 404 sin cabeceras CORS parecía un error de CORS
Juni sin partidos	Los 3 partidos de ejemplo eran de octubre y ya habían pasado	Se agregaron 3 partidos futuros (22 de noviembre, 6 y 13 de diciembre) al seed y se migró la base

10. Qué hay corriendo y cuánto cuesta

Recurso	Detalle	Costo
RDS MySQL jfc-mysql	db.t4g.micro, MySQL 8.4.9	Por hora: apagar al terminar
5 Lambdas jfc-*	personalización, agente, escáner, notificador y migración	Por uso
API Gateway junior-fc-api	10 rutas, autorizador JWT de Cognito	Por uso
SQS: jfc-avisos y jfc-avisos-fallidos	Cola principal y cola de fallos	Por uso
EventBridge	Regla rate(1 minute)	Por uso
OpenShift Developer Sandbox	Deployment boleteria con 2 pods	Gratis
API de Claude	Haiku 5.5 (moderación) y Sonnet 5.5 (agente)	Por token

Para verlo en AWS: CloudFormation, stack junior-fc-backend, pestaña Resources (45 recursos) y la opción «View in Infrastructure Composer» de la pestaña Template, que dibuja el diagrama de lo desplegado. Cost Explorer muestra el gasto por servicio.

11. Cómo probarlo

- Entrar a la tienda por la página de login (CloudFront) y abrir la camiseta Local 2026 para personalizarla.
- **IA:** probar GARCÍA o URIBE (se guardan) y MILLONARIOS, NIKE, ASDFGH, M13RD4 (se bloquean con un mensaje fijo). Usar un número distinto de 12, que está reservado.

- **Agente:** abrir Juni y escribir «quiero 2 boletas para el próximo partido». Debe listar los partidos, preguntar tribuna y cantidad, reservar y pedir confirmación.
- **Asíncrono:** reservar con Juni sin confirmar; cuando falten 2 minutos aparece el aviso. Verlo en vivo con `aws logs tail /aws/lambda/jfc-avisos-notificador --follow`.

12. Pendientes y límites

- Configurar el destino del aviso: remitente verificado en SES (la cuenta está en sandbox) y URL del webhook del grupo de la clase en `.secrets/jfc.env`, y volver a desplegar.
- Ejecutar la prueba completa del aviso asíncrono con una reserva real.
- La clave de la API de Claude vive en una variable de entorno de las Lambdas: es aceptable para esta actividad; en producción iría en Secrets Manager.
- Apagar RDS y escalar boleteria a 0 pods al terminar la presentación.